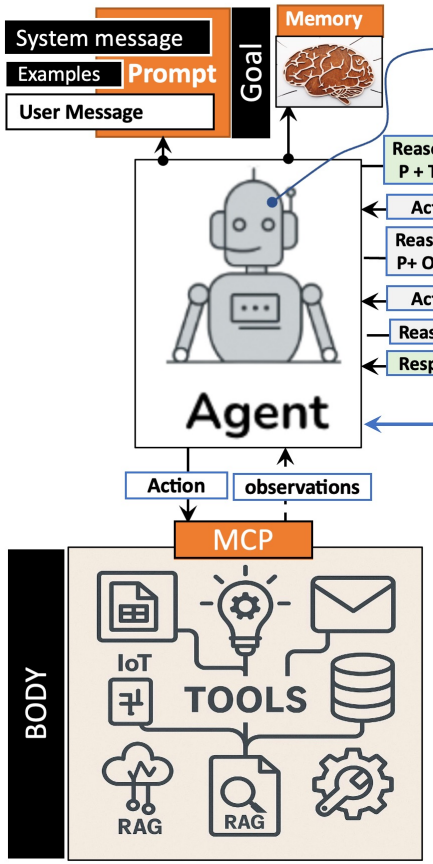
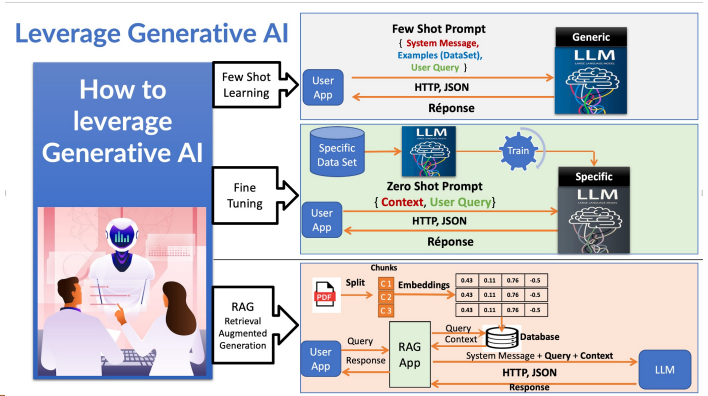
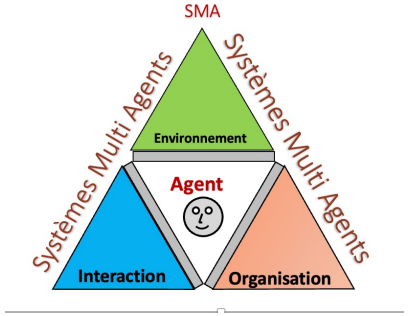
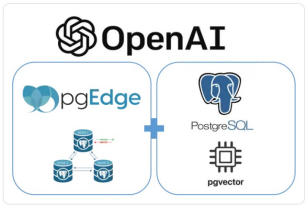
Agentic AI – Multi Agent Systems



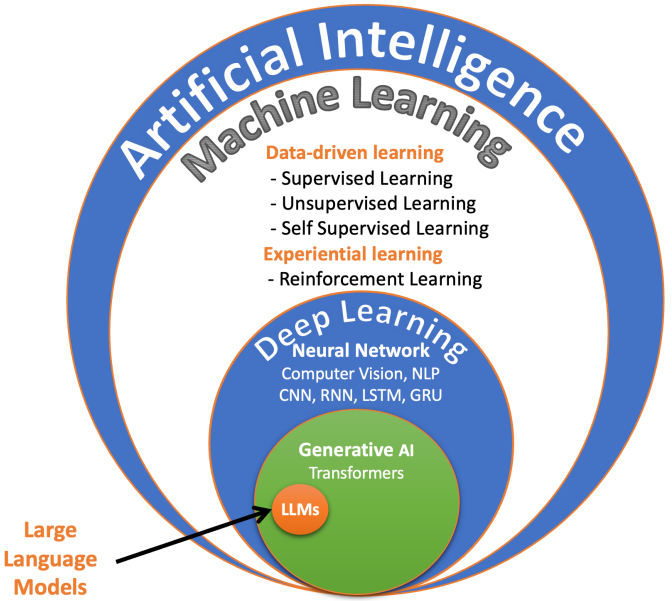












Retrieval Augmented Generation
RAG

Leverage Generative AI

How to leverage Generative AI



Few Shot Learning

Fine Tuning

RAG
Retrieval
Augmented
Generation

Few Shot Prompt
{ System Message,
Examples (DataSet),
User Query }

User App

HTTP, JSON

Réponse

Generic LLM

Specific Data Set

LLM

Train

Specific LLM

Zero Shot Prompt
{ Context, User Query}

User App

HTTP, JSON

Réponse

Chunks

PDF

Split

C 1
C 2
C 3

Embeddings

0.43	0.11	0.76	-0.5
0.43	0.11	0.76	-0.5
0.43	0.11	0.76	-0.5

Database

Query
Context

User App

Query

Response

RAG App

System Message + **Query** + **Context**

HTTP, JSON

Response

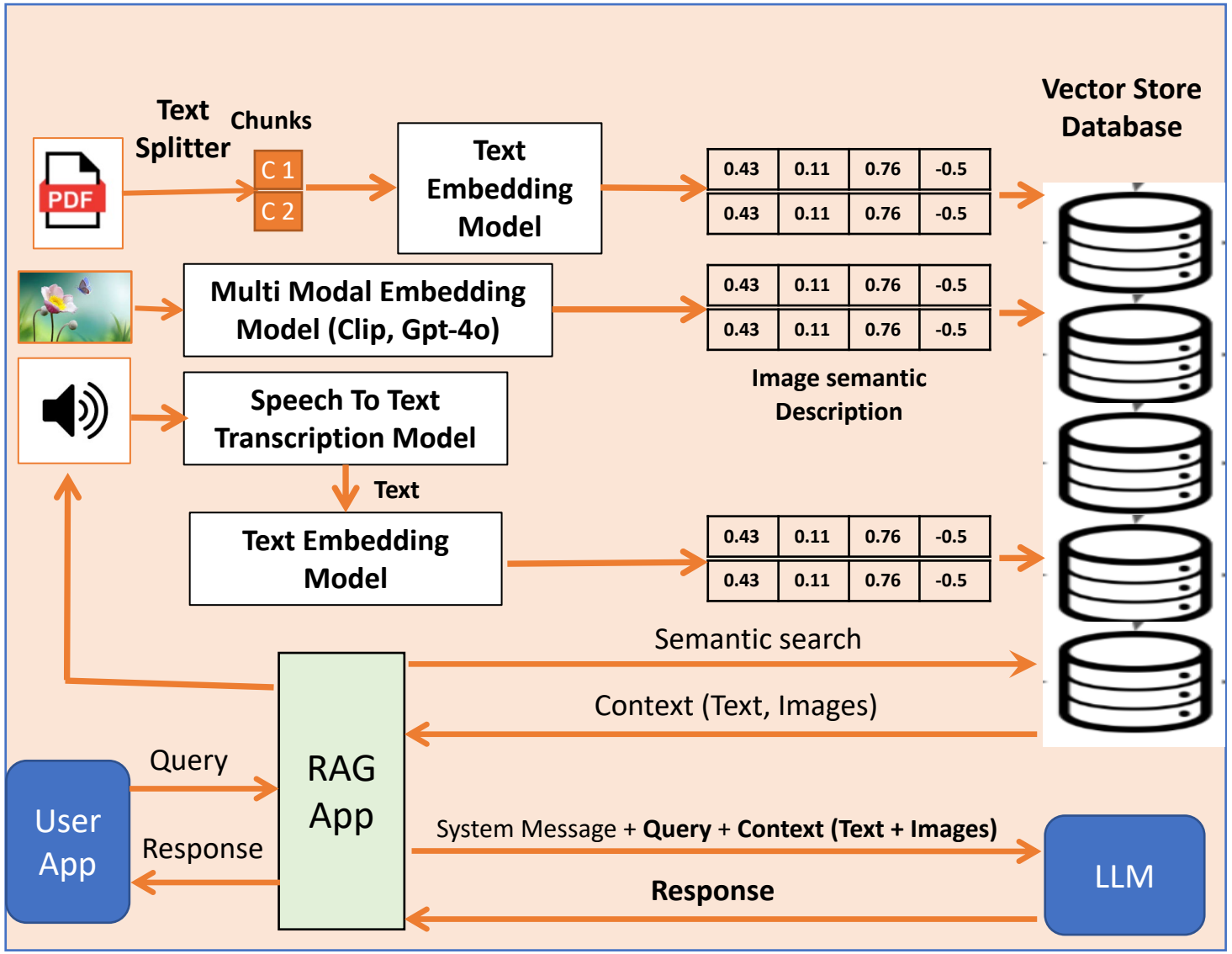
LLM

Multi Modal Retrieval Augmented Generation

Multi Modal RAG



Multi RAG Retrieval Augmented Generation



Step 1: Creating a Vector Database (Indexing)

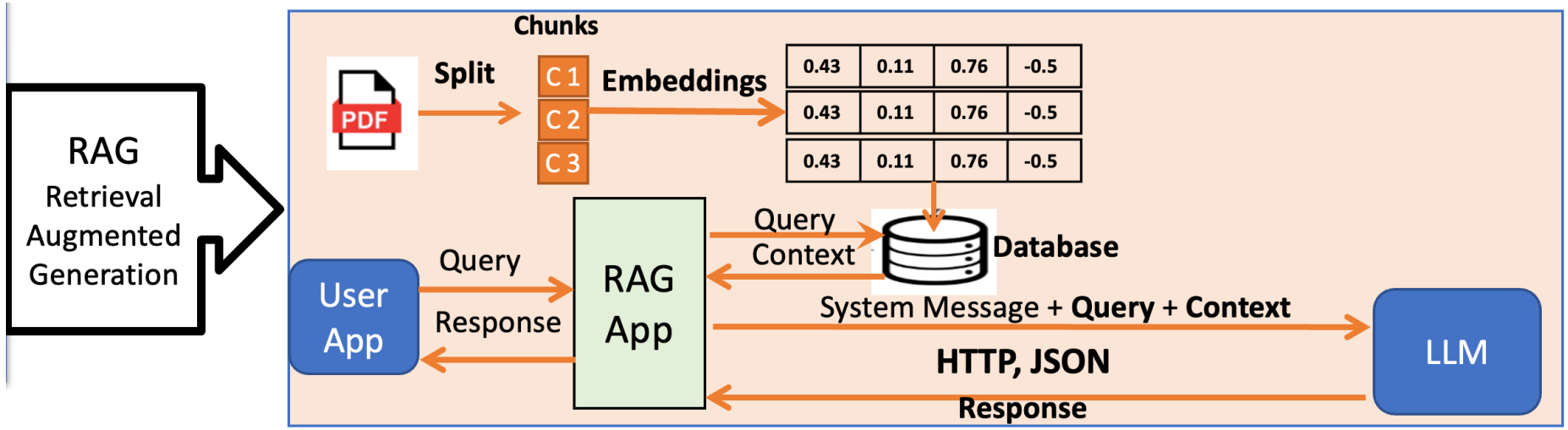
- Nous créons une **base de données vectorielle** en découpant le document d'entrée en chunks, puis en associant à chaque chunk un **vecteur** à l'aide d'un modèle d'embeddings.
- Le processus global est décrit dans la figure ci-dessous.
- Une bonne méthode pour choisir un modèle d'embedding consiste à utiliser le [classement des embeddings](https://huggingface.co/spaces/mteb/leaderboard) et à sélectionner un modèle open source performant pour le cas d'usage visé.
- Un bon modèle d'embeddings **polyvalent** est [`**gte-large**`](https://huggingface.co/thenlper/gte-large).

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFDirectoryLoader, PyPDFLoader
from langchain_community.document_loaders import PyPDFDirectoryLoader
from langchain_community.vectorstores import Chroma
from dotenv.ipython import load_dotenv
import os
```

```
pdf_file = "./pdfs/agentic-ai-foundations.pdf"
```

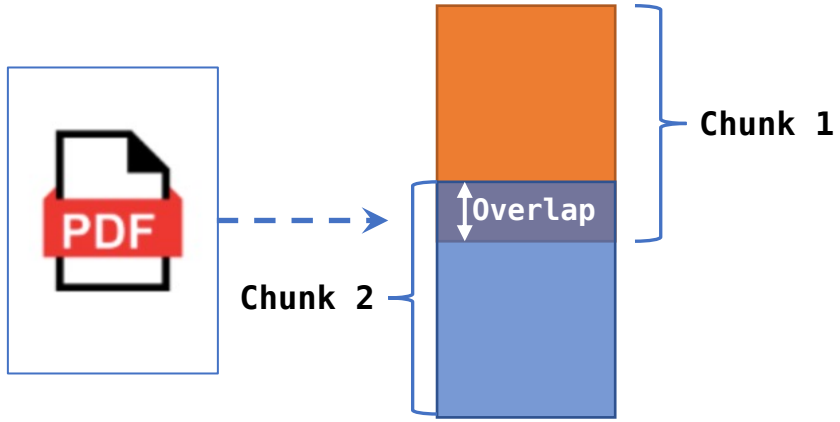
```
pdf_loader = PyPDFLoader(pdf_file)
```

```
loader = PyPDFDirectoryLoader(path = "./pdfs")
```



Chunking documents

- Une fois le choix du modèle d'embeddings effectué, nous pouvons lui fournir les documents en entrée.
- Cependant, comme les documents contiennent plusieurs pages (y compris des figures et des tableaux), il nous faut une méthode pour **parser** des portions individuelles du document, compatibles avec le **modèle d'embedding**.
- Nous savons également que le modèle d'embeddings ne peut pas traiter de texte au-delà d'une longueur de contexte fixe (**512 tokens** pour `**gte-large**`).
- Cela correspond approximativement à **400 mots** ou à environ **une page** de texte.
- En gardant cette contrainte à l'esprit, nous allons transformer le fichier PDF en **chunks** de texte ne dépassant pas **512 tokens**.




```
text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    encoding_name='cl100k_base',
    chunk_size=512,
    chunk_overlap=16
)
```

```
chunks = loader.load_and_split(text_splitter)
```

- Une autre approche consisterait à découper le fichier par **sections** du rapport (par ex., facteurs de risque, procédures judiciaires, déclarations de sécurité).
- Une stratégie de **chunking** courante consiste à utiliser des **chunks de taille fixe** (telle que définie par le modèle d'embedding) et à appliquer un **léger chevauchement (Overlap)** entre les chunks
- L'utilisation d'un **chevauchement (Overlap)** permet d'assurer la continuité entre les chunks et de récupérer des chunks pertinents même lorsque l'information liée à la requête se trouve soit au début, soit à la fin du chunk.

Vector Store - ChromaDB, Embeddings

```
from langchain_openai import OpenAIEmbeddings
embedding_model = OpenAIEmbeddings(model='text-embedding-ada-002')
```

```
vectorstore = Chroma.from_documents(
  chunks,
  embedding_model,
  collection_name="Agentic_AI"
)
```

Model	~ Pages per dollar	Performance on MTEB eval	Max input
text-embedding-3-small	62,500	62.3%	8192
text-embedding-3-large	9,615	64.6%	8192
text-embedding-ada-002	12,500	61.0%	8192

```
retriever = vectorstore.as_retriever(
  search_type='similarity',
  search_kwargs={'k': 5}
)
```

```
retrieved_chunks=retriever.invoke("Quels sont les fonctions d'un agent?")
```

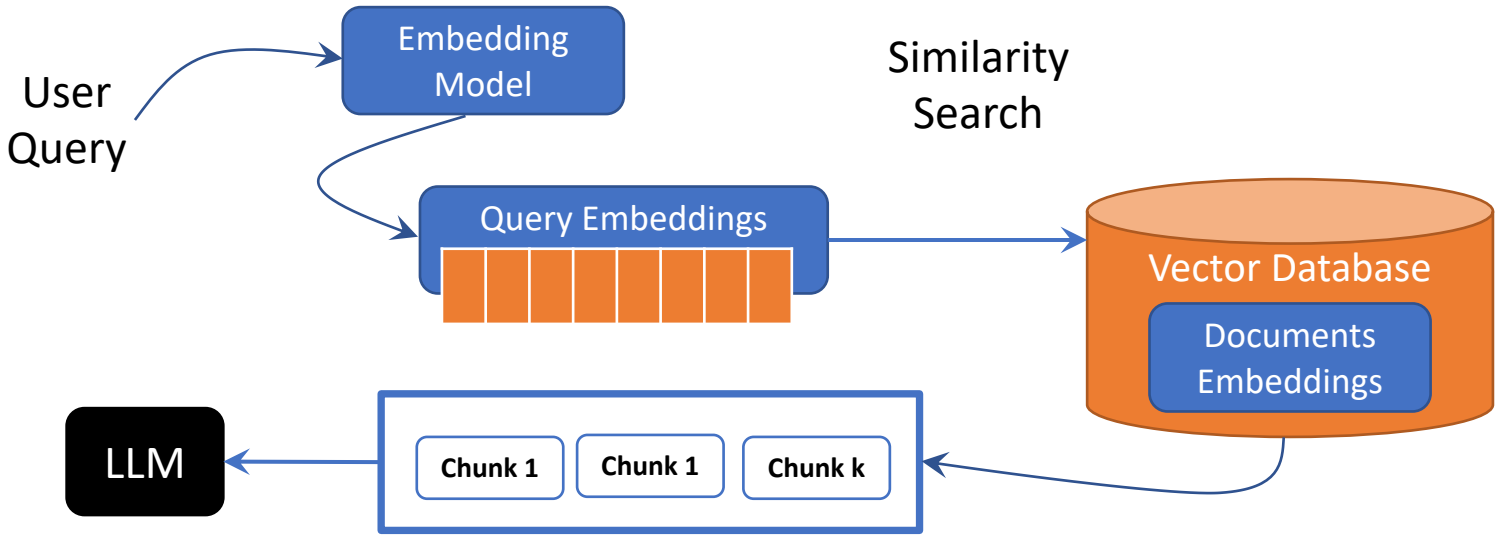
```
print(retrieved_chunks)
```

```
[Document(metadata={'page_label': '10', 'page': 14, 'creationdate': '2026-02-20T10:24:07+00:00', 'source': 'pdfs/agentic-ai-foundations.pdf', 'title': "AWS Directives prescriptives - Les fondements de l'IA agentique sur AWS", 'author': 'AWS', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'producer': 'Apache FOP Version 2.6', 'total_pages': 90, 'keywords': "…."}, page_content="….."),
Document(….), Document(….), Document(….), Document(….)]
```

```
print(len(retrieved_chunks))
```

Step 2: Querying the Vector Database

- Une fois la **base de données vectorielle** mise en place, l'embedding généré à partir du texte de la requête est comparé à l'ensemble des embeddings de documents présents dans la base de données vectorielle.
- Les documents les plus similaires sont ensuite récupérés et présentés, avec la requête d'origine, dans le prompt.
- Enfin, dans le prompt, nous demandons au LLM de se concentrer uniquement sur le **contexte** pour répondre à la requête de l'utilisateur.
- L'idée clé ici est que la requête **et** le contexte sont tous deux **injectés dynamiquement** dans le prompt.




```
prompt_template = """

Answer the following question based
only on provided context
The context is delimited by <context>
tag
The user question is delimited by
<question> tag
If the answer is not found in the
context, answer : I DO NOT KNOW
<context>
 {context}
</context>
<question>
 {question}
</question>

"""
```

Step 3: Querying and Generation

```
query ="Nom, prénom et Liste des diplômes obtenus"
```

```
relevant_document_chunks = retriever.invoke(query)
context_list = [d.page_content for d in relevant_document_chunks]
context_for_query = ". ".join(context_list)
```

```
prompt = prompt_template.format(context=context_for_query, question=query)
```

```
resp = llm.invoke(prompt)
```

```
from IPython.display import Markdown
```

```
print(display(Markdown(resp.content)))
```

Nom, prénom : Mohamed Youssfi

Liste des diplômes obtenus :

- Doctorat d'Etat, Fac des sciences Rabat (2015)
- Doctorat de Troisième cycle, Fac des sciences Rabat (1996)
- Diplôme des Etudes Approfondies à la fac des sciences de Rabat (1994)
- Diplôme de Professeur Second Cycle de l'ENSET Mohammedia (1993)

```
prompt_template = """

Answer the following question based only
on provided context
The context is delimited by <context>
tag
The user question is delimited by
<question> tag
If the answer is not found in the
context, answer : I DO NOT KNOW
<context>
 {context}
</context>
<question>
 {question}
</question>
"""
```

Defining the RAG function for response

```
def RAG(query, llm=llm, prompt_template=prompt_template):
    context_docs = retriever.invoke(query)
    context_list = [d.page_content for d in context_docs]
    context_for_query = ". ".join(context_list)
    prompt = prompt_template.format(context=context_for_query, question=query)
    resp=llm.invoke(prompt)
    return resp.content
```

```
response = RAG("Fonction actuelle du Pr YOUSSFI")
print(display(Markdown(response)))
```

4 - Evaluation

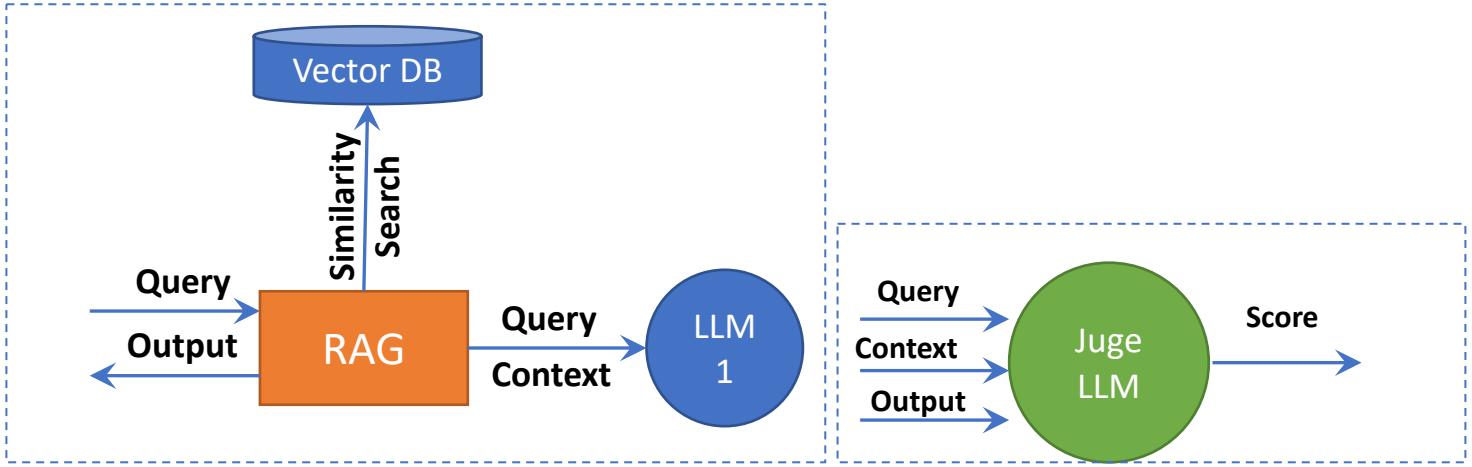
- Utilisons maintenant la méthode **LLM-as-a-judge** pour vérifier la qualité du système RAG selon deux paramètres : la récupération (**retrieval**) et la génération (**generation**). Nous illustrons cette évaluation à partir des réponses générées à la question de la section précédente.
- Nous utilisons le même modèle pour l'évaluation ; en d'autres termes, ici le LLM s'auto-évalue en notant la qualité de sa propre performance dans la tâche.

```
user_input ="État du résultat global consolidé"
```

```
relevant_document_chunks = retriever.invoke(user_input)
context_list = [d.page_content for d in relevant_document_chunks]
context_for_query = ". ".join(context_list)
```


```
relevant_document_chunks = retriever.invoke(user_input)
context_list = [d.page_content for d in relevant_document_chunks]
context_for_query = ". ".join(context_list)
```


```
user_message_template = """
 ###Question
 {question}
 ###Context
 {context}
 ###Answer
 {answer}
"""
```



4 - Evaluation

```
groundedness_rater_system_message="""
Vous êtes chargé d'évaluer des réponses générées par une IA à des questions posées par des utilisateurs.
On vous présentera une question, le contexte utilisé par le système d'IA pour générer la réponse, ainsi qu'une réponse
générée par l'IA à la question.

Dans l'entrée, la question commencera par ###Question, le contexte commencera par ###Context, et la réponse générée par
l'IA commencera par ###Answer.

Critères d'évaluation :
La tâche consiste à juger dans quelle mesure la réponse respecte la métrique.

1 – La métrique n'est pas respectée du tout
2 – La métrique n'est respectée que dans une mesure limitée
3 – La métrique est respectée dans une bonne mesure
4 – La métrique est respectée en grande partie
5 – La métrique est entièrement respectée

Métrique :
La réponse doit être dérivée uniquement des informations présentées dans le contexte.

Instructions :

Écrivez d'abord les étapes nécessaires pour évaluer la réponse selon la métrique.
Donnez une explication étape par étape indiquant si la réponse respecte la métrique, en considérant la question et le
contexte comme entrées.
Évaluez ensuite dans quelle mesure la métrique est respectée.
Utilisez les informations précédentes pour noter la réponse selon les critères d'évaluation et attribuer un score.
"""
```

4 - Evaluation

```
def evaluate(system_message,user_message_template, question, model=llm):
    retrieved_chunks=retriever.invoke(question)
    context_list = [d.page_content for d in retrieved_chunks]
    context = ". ".join(context_list)
    answer = RAG(question)
    prompt = f"""
     {system_message}\n
     USER:
     {user_message_template.format(question=question, context=context, answer=answer)}
    """
    juge_response= model.invoke(prompt)
    return juge_response.content
```

4 - Evaluation

```
resp=evaluate(groundedness_rater_system_message, user_message_template, user_input)
print(display(Markdown(resp)))
```

Pour évaluer la réponse selon la métrique, voici les étapes nécessaires :

1. **Identifier la question** : La question demande l'état du résultat global consolidé.
2. **Analyser le contexte** : Le contexte fournit des informations financières consolidées pour le Groupe OCP au 31 décembre 2023, y compris le résultat global consolidé pour l'exercice 2023.
3. **Extraire les informations pertinentes du contexte** :
   - Le résultat global consolidé pour l'exercice 2023 est de 14 187 millions de dirhams.
   - La répartition est la suivante : part du Groupe est de 14 259 millions de dirhams et part des intérêts ne donnant pas le contrôle est de (72) millions de dirhams.
4. **Comparer la réponse avec les informations du contexte** :
   - La réponse indique que le résultat global consolidé pour l'exercice 2023 est de 14 187 millions de dirhams, ce qui correspond à l'information du contexte.
   - La répartition mentionnée dans la réponse (part du Groupe 14 259 millions de dirhams et part des intérêts ne donnant pas le contrôle (72) millions de dirhams) est également conforme aux données fournies dans le contexte.
5. **Évaluer la conformité de la réponse à la métrique** : La réponse est entièrement dérivée des informations présentées dans le contexte. Elle reprend fidèlement les chiffres et la répartition du résultat global consolidé tels qu'ils apparaissent dans le contexte.

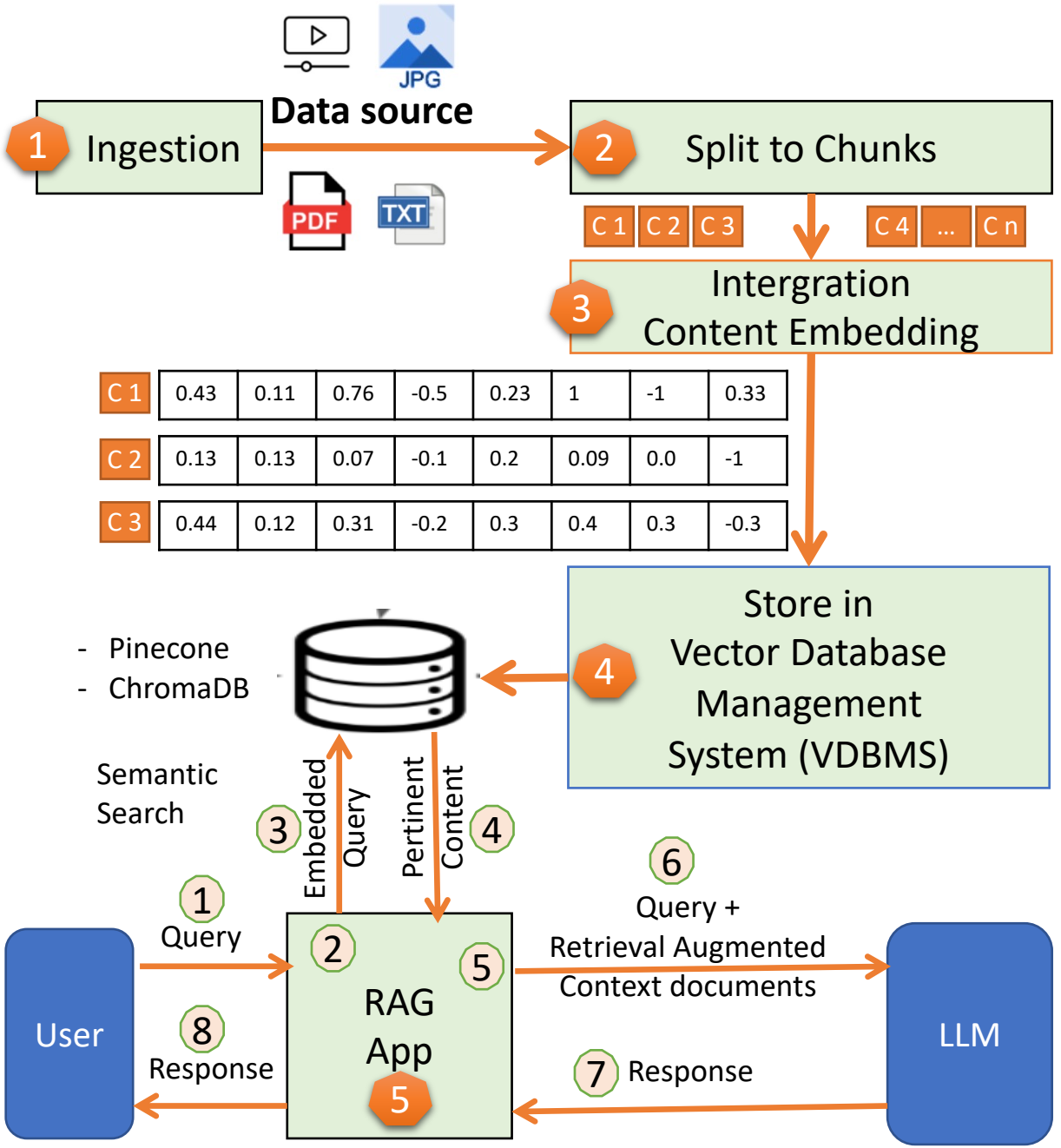
Évaluation : La réponse respecte entièrement la métrique, car elle est directement et uniquement dérivée des informations fournies dans le contexte.

Score : 5

The building blocks of RAG

Step 5 : RAG App

- Développer une application qui permet de :
  1. Saisir la requête utilisateur
  2. Transformer la requête en représentation vectorielle
  3. Interroger le VDBMS (Pinecone, ChromaDB, PostGreSQL) pour faire une recherche sémantique.
  4. Retrouver les documents pertinents lié au contexte de la requête
  5. Injecter les contenus contextuels retrouvés dans le prompt
  6. Présenter ce prompt au LLM pour l'inciter à réponse à la requête utilisateur en utilisant les données contextuelles les plus pertinentes.
  7. Récupérer la réponse du LLM
  8. Envoyer à l'utilisateur la réponse

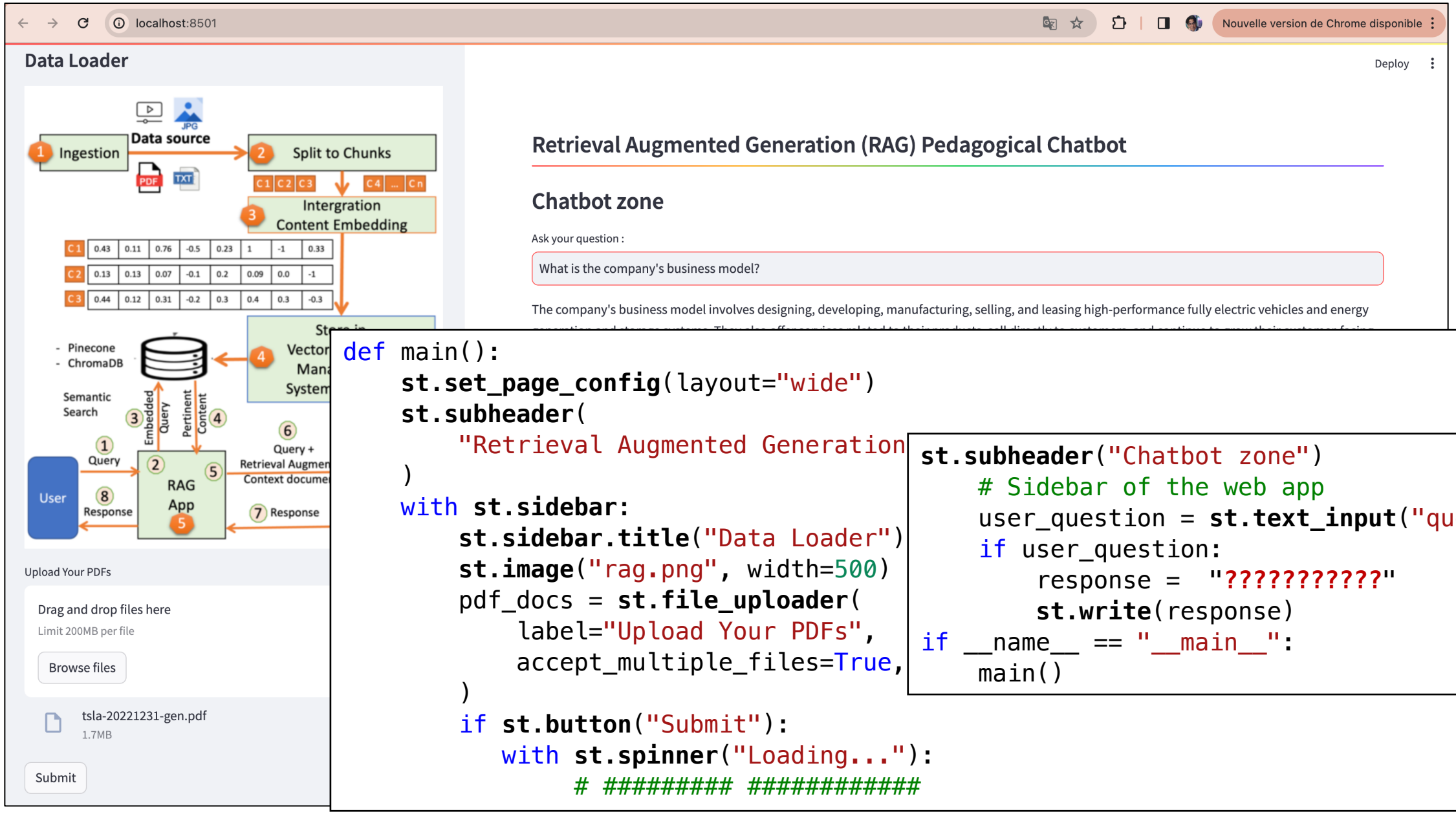


The building blocks of RAG

```
import streamlit as st
from PyPDF2 import PdfReader
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from dotenv.ipython import load_dotenv
from langchain_openai import ChatOpenAI
```

```
load_dotenv(override=True)
from langchain_text_splitters import (
    CharacterTextSplitter,
    RecursiveCharacterTextSplitter,
)
prompt_template = """
Answer the following question based only on the provided context:
<context>
    {context}
</context>
<question>
    {input}
</quesrtion>
"""
llm = ChatOpenAI(model="gpt-4o", temperature=0)
```

The building blocks of RAG




```python
def main():
    st.set_page_config(layout="wide")
    st.subheader(
        "Retrieval Augmented Generation
    )
    with st.sidebar:
        st.sidebar.title("Data Loader")
        st.image("rag.png", width=500)
        pdf_docs = st.file_uploader(
            label="Upload Your PDFs",
            accept_multiple_files=True,
        )
        if st.button("Submit"):
            with st.spinner("Loading..."):
                # ######### ############
```

```python
st.subheader("Chatbot zone")
    # Sidebar of the web app
    user_question = st.text_input("question :")
    if user_question:
        response =  "???????????"
        st.write(response)
if __name__ == "__main__":
    main()
```

RAG : Extracting Text from PDFs

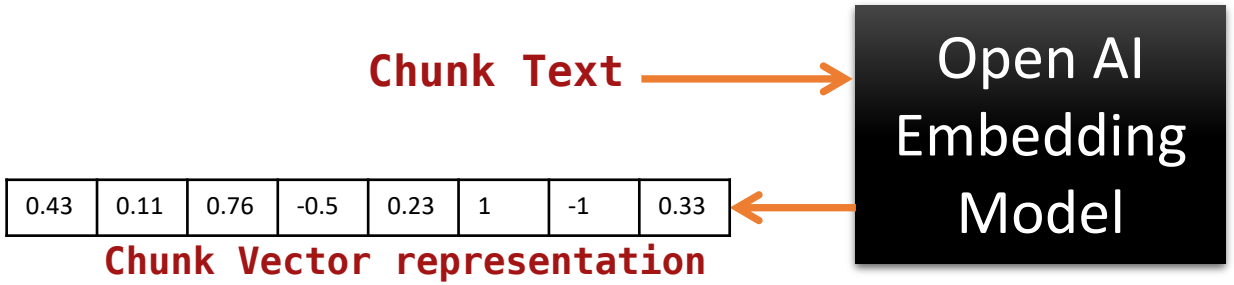
```
with st.sidebar:
    st.sidebar.title("Data loader")
    st.image("rag.png")
    pdf_docs = st.file_uploader(label="Load your pdfs", accept_multiple_files=True)
    if st.button("Submit"):
        with st.spinner("Loading"):
            content = ""
            for pdf in pdf_docs:
                reader = PdfReader(pdf)
                for page in reader.pages:
                    content += page.extract_text()

            splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
                chunk_size=512, chunk_overlap=16
            )

            chunks = splitter.split_text(content)
            st.write(chunks)

            embedding_model = OpenAIEmbeddings()
            vector_store = Chroma.from_texts(
                chunks,
                embedding_model,
                collection_name="data_collection",
            )
            retriever = vector_store.as_retriever(
                kwargs={"k": 5},
            )

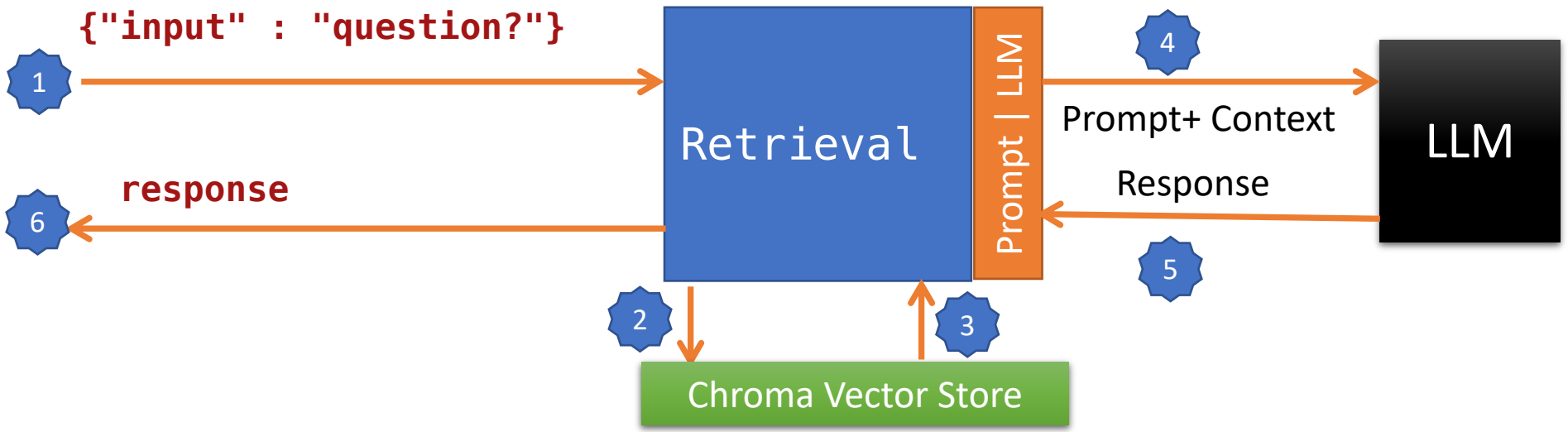
            st.session_state.retriever = retriever
```



RAG

```
st.subheader("Chatbot")
user_question = st.text_input("Ask Your Question")
if user_question:
    context_docs = st.session_state.retriever.invoke(user_question)
    context_list = [d.page_content for d in context_docs]
    context_text = ". ".join(context_list)
    # st.write(context_text)
    prompt = prompt_template.format(context=context_text, input=user_question)
    resp = llm.invoke(prompt)
    st.write(resp.content)
```

1. User Query
2. Similarity Vector Search
3. Relevant Documents
4. Prompt LLM
5. LLM Response
6. RAG Response



```python
import streamlit as st
from PyPDF2 import PdfReader
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from dotenv.ipython import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv(override=True)
from langchain_text_splitters import (
    CharacterTextSplitter,
    RecursiveCharacterTextSplitter,
)

prompt_template = """
Answer the following question based only on the provided context:
<context>
    {context}
</context>
<question>
    {input}
</quesrtion>
"""

llm = ChatOpenAI(model="gpt-4o", temperature=0)
```

```python
def main():
    st.set_page_config(page_title="RAG", layout="wide")
    st.subheader("Retrieval Augmented generation", divider="blue")

    with st.sidebar:
        st.sidebar.title("Data loader")
        st.image("rag.png")
        pdf_docs = st.file_uploader(label="Load your pdfs", accept_multiple_files=True)
        if st.button("Submit"):
            with st.spinner("Loading"):
                content = ""
                for pdf in pdf_docs:
                    reader = PdfReader(pdf)
                    for page in reader.pages:
                        content += page.extract_text()

                splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
                    chunk_size=512, chunk_overlap=16
                )

                chunks = splitter.split_text(content)
                st.write(chunks)

                embedding_model = OpenAIEmbeddings()
                vector_store = Chroma.from_texts(
                    chunks,
                    embedding_model,
                    collection_name="data_collection",
                )
                retriever = vector_store.as_retriever(
                    kwargs={"k": 5},
                )

                st.session_state.retriever = retriever
    st.subheader("Chatbot")
    user_question = st.text_input("Ask Your Question")
    if user_question:
        context_docs = st.session_state.retriever.invoke(user_question)
        context_list = [d.page_content for d in context_docs]
        context_text = ". ".join(context_list)
        # st.write(context_text)
        prompt = prompt_template.format(context=context_text, input=user_question)

        resp = llm.invoke(prompt)

        st.write(resp.content)
if __name__ == "__main__":
    main()
```